Recruitment Management System

Data Flow Diagrams — Levels 0, 1 and 2

Context Diagram · Nine Major Processes · Four Level-2 Decompositions

Process Catalogue · Data Store Catalogue · Data Dictionary

Balancing Rules · Interview Preparation

Gane–Sarson notation · Rendered with Graphviz

1. What a DFD Is — and How It Differs From Your ER Diagram

An Entity-Relationship diagram answers **"what does the system store?"** A Data Flow Diagram answers **"how does data move, and who transforms it?"** They describe the same system from perpendicular angles, and examiners frequently ask you to explain the difference.

Aspect	ER Diagram	Data Flow Diagram
Question answered	What is stored, and how is it related?	What moves, and what transforms it?
View	Static — structure at rest	Dynamic — behaviour over time
Building blocks	Entities, attributes, relationships	Processes, data flows, data stores, external entities
Answers	"Where does the bid amount live?"	"How does a bid become a funded contract?"
Maps to	Database tables	Services, modules, API endpoints

The one-line answer to have ready: "The ER diagram is the noun view — tables and their relationships. The DFD is the verb view — processes and the data moving between them. The data stores in my DFD map directly onto groups of tables in my ER diagram, which is how the two stay consistent."

1.2 The four symbols

External Entity (square) — a person or system outside the boundary that supplies or receives data. The system cannot control it. Here: candidates, recruiters, admins, the payment gateway, the email service, cloud storage.

Process (rounded box, numbered) — work that transforms input data into output data. Every process must have at least one input and one output.

Data Store (open-ended box, labelled D1…D12) — data at rest. A store is passive: it never transforms anything, it only holds.

Data Flow (labelled arrow) — a packet of data in motion. The label names *what* is moving, never how.

1.3 Rules the diagrams follow

- **No black holes.** Every process has at least one input and one output. A process that only consumes data is an error.
- **No miracles.** A process cannot output data it was never given.
- **No store-to-store flows.** Data cannot move between two data stores directly — a process must move it, because stores are passive.
- **No entity-to-entity flows.** If two external entities exchange data without the system touching it, it is outside the boundary and does not belong on the diagram.
- **No entity-to-store flows.** An external entity can never read or write a store directly; a process must mediate.
- **Balancing.** The inputs and outputs of a parent process must exactly match those of its child diagram. Level 1's inputs and outputs must equal Level 0's.

The mistake examiners look for first is a flow drawn directly between two data stores, or between an external entity and a data store. Both are formally invalid. The second most common is drawing control flow — "if approved then..." — which belongs in a flowchart, not a DFD. A DFD shows *data*, not sequence or decisions.

2. Level 0 — Context Diagram

The whole system as a single process, numbered 0, showing only what crosses the system boundary. Its purpose is to fix scope: everything inside the circle is yours to build, everything outside is not.

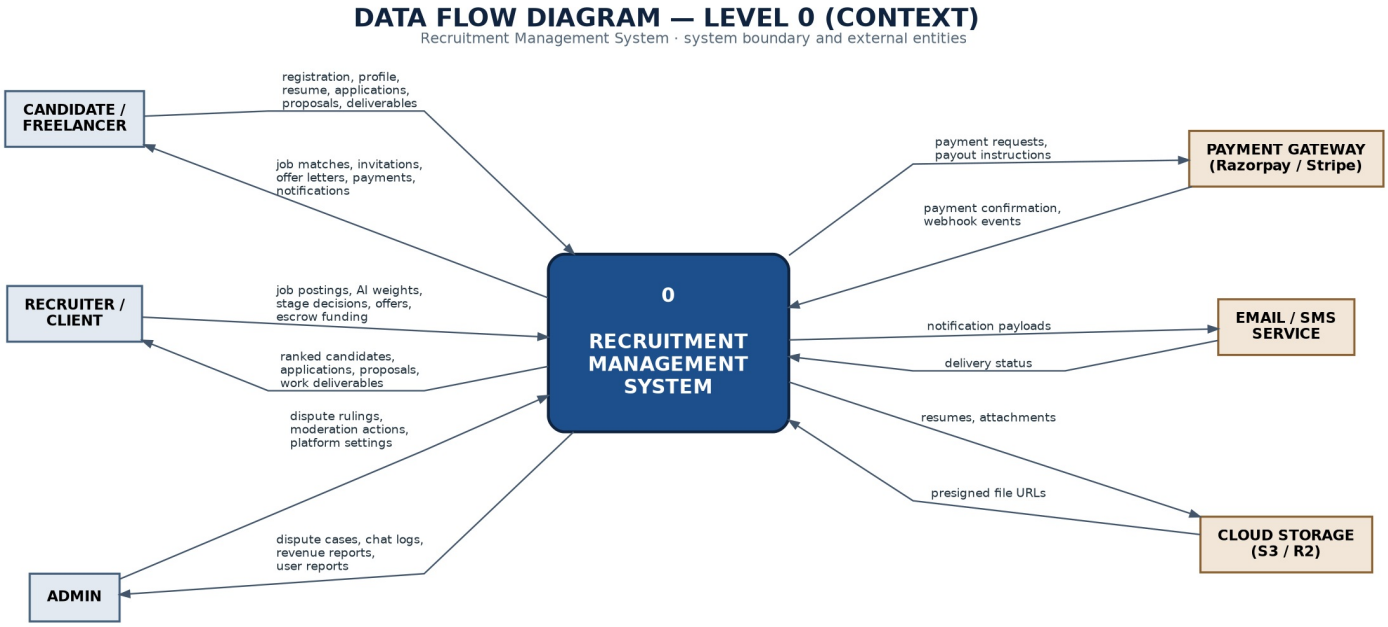


Figure 1 — Level 0 Context Diagram: six external entities, one process, no data stores

Note that a context diagram shows **no data stores**. All storage is internal to the system, so at this level of abstraction it is invisible. Being asked "why are there no data stores in your Level 0?" is a common check, and the answer is exactly that.

2.1 External entities and why each is outside the boundary

Entity	Sends to system	Receives from system
Candidate / Freelancer	Registration, profile, resume, skills, applications, proposals, milestone deliverables, reviews	Job matches, invitations, offer letters, payment credits, notifications
Recruiter / Client	Job descriptions, AI weightings, stage configuration, hiring decisions, escrow funding, approvals	Ranked candidate shortlists, applications, proposals, delivered work, invoices
Admin	Dispute rulings, ban and suspension actions, platform settings	Dispute cases with evidence, revenue reports, user reports
Payment Gateway	Payment confirmations, webhook events, payout status	Charge requests, payout instructions
Email / SMS Service	Delivery status	Notification payloads
Cloud Storage	Presigned URLs, stored file references	Resumes, deliverables, dispute evidence

The last three are *external systems* rather than people. They belong on the diagram because data crosses the boundary to reach them, and you do not control their internals. Distinguishing human actors from external systems while treating both as external entities is a point worth making explicitly.

3. Level 1 — Major Processes

Process 0 decomposed into the nine processes that constitute the system, now with the twelve data stores made visible. Every flow crossing the boundary in Level 0 reappears here, attached to a specific process — that correspondence is what "balancing" means.

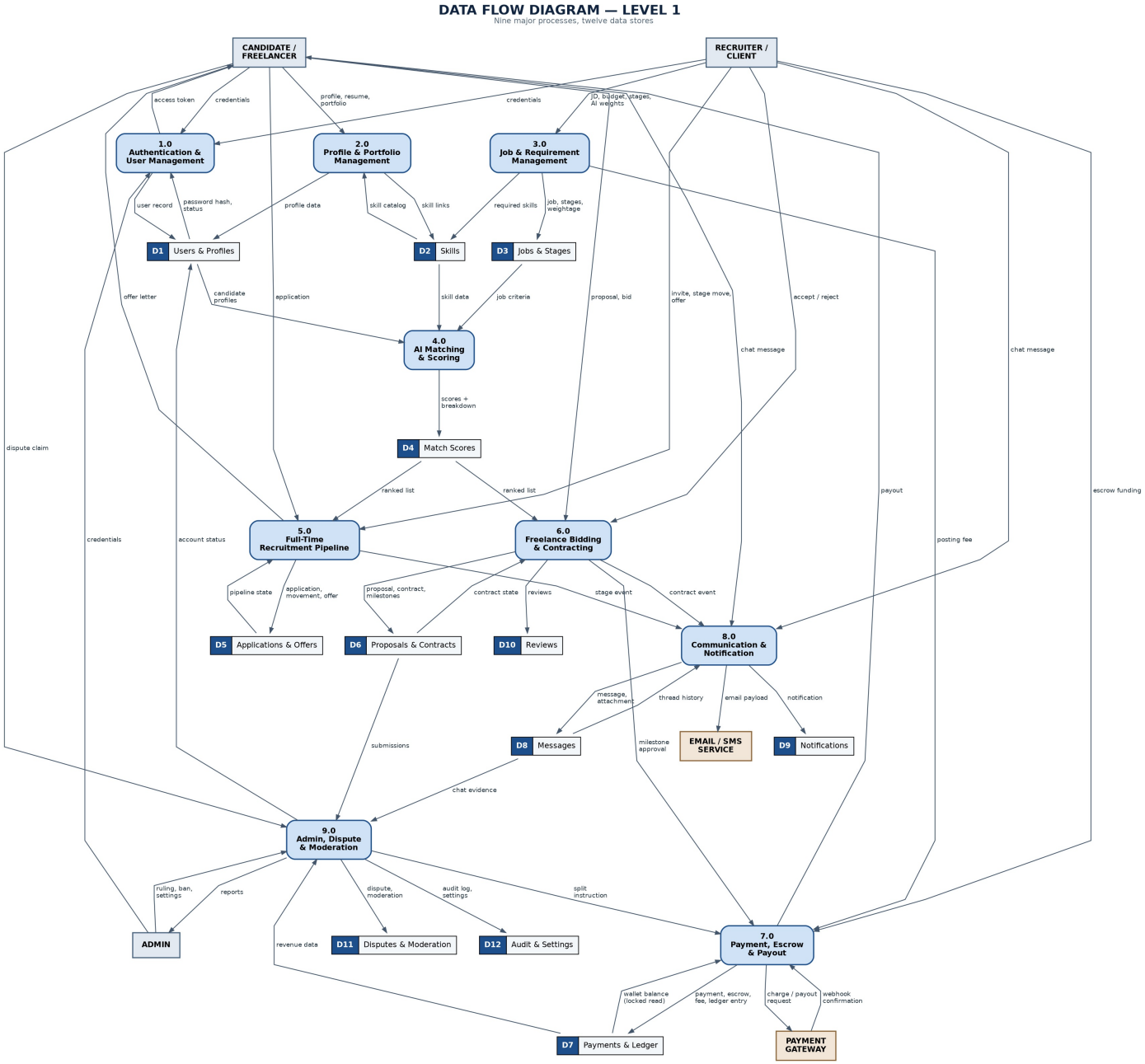


Figure 2 — Level 1: nine processes, twelve data stores, five external entities

3.1 Process catalogue

No.	Process	Responsibility
1.0	Authentication & User Management	Registration, login, token issue and refresh, role assignment, account status checks
2.0	Profile & Portfolio Management	Candidate profiles, resume versions, freelance portfolios, skill declaration, recruiter and company profiles
3.0	Job & Requirement Management	Job creation for both types, required skills, hiring-stage configuration, AI weightage setup, posting-fee trigger
4.0	AI Matching & Scoring	Pre-filtering, feature extraction, weighted scoring, persistence of score plus explainability breakdown, ranking
5.0	Full-Time Recruitment Pipeline	Invitations, applications, stage movement with history, interview scheduling and feedback, offers
6.0	Freelance Bidding & Contracting	Proposals, acceptance, contract and milestone creation, work submission, approval, closure and reviews
7.0	Payment, Escrow & Payout	Platform fees, escrow funding and release, gateway reconciliation, wallet ledger, withdrawals
8.0	Communication & Notification	Contract- and application-scoped chat, message persistence, notification generation and dispatch
9.0	Admin, Dispute & Moderation	Dispute intake and arbitration, fund-split instruction, user moderation, platform settings, audit and revenue reporting

3.2 Data store catalogue — and the ER tables behind each

This mapping is the bridge between your two diagrams. If asked how they stay consistent, this table is the answer.

Store	Name	ER tables it represents
D1	Users & Profiles	`users`, `candidate_profiles`, `recruiter_profiles`, `admin_profiles`, `companies`, `resumes`, `freelance_portfolios`, `portfolio_projects`
D2	Skills	`skills`, `candidate_skills`, `job_skills`, `portfolio_project_skills`
D3	Jobs & Stages	`jobs`, `hiring_stages`, `ai_weightages`
D4	Match Scores	`match_scores`
D5	Applications & Offers	`applications`, `stage_movements`, `invitations`, `interviews`, `offers`
D6	Proposals & Contracts	`proposals`, `contracts`, `milestones`, `milestone_submissions`
D7	Payments & Ledger	`payments`, `escrows`, `platform_fees`, `wallets`, `transactions`, `payouts`
D8	Messages	`chat_rooms`, `chat_participants`, `messages`, `attachments`
D9	Notifications	`notifications`
D10	Reviews	`reviews`
D11	Disputes & Moderation	`disputes`, `moderation_actions`
D12	Audit & Settings	`audit_logs`, `platform_settings`

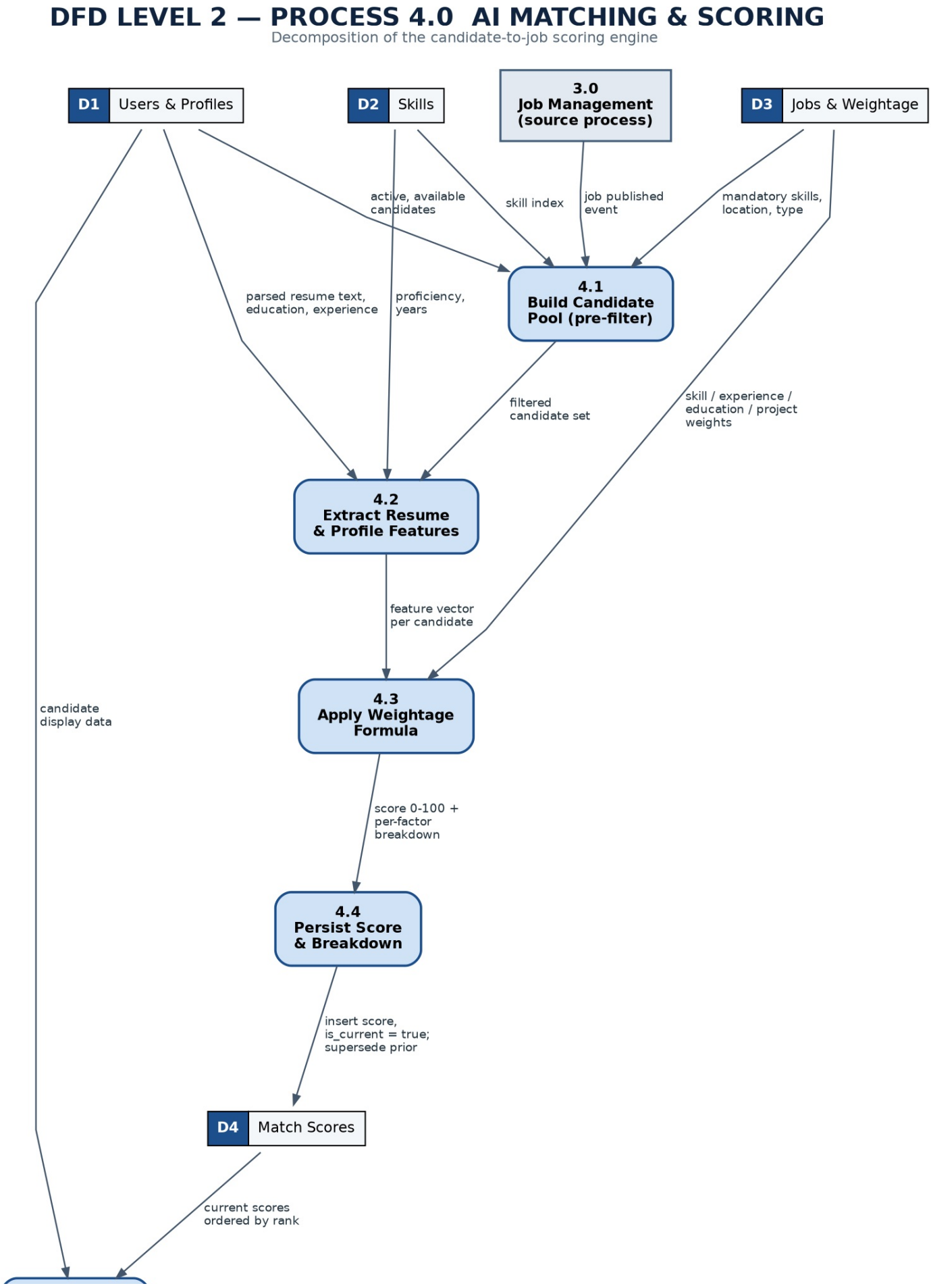
> **Why data stores group several tables.** A DFD store is a logical collection, not a physical table. D1 covers eight tables because, from the perspective of data movement, "user and profile data" behaves as one unit — processes read and write it as a whole. Splitting D1 into eight stores would add no information about flow and would make the diagram unreadable. Being able to justify the grouping is better than apologising for it.

4. Level 2 — Process Decompositions

Level 2 explodes a single Level 1 process into its constituent sub-processes, numbered `4.1`, `4.2` and so on. Not every process needs decomposing — you decompose the ones with real internal complexity. Four are shown here; the remainder are simple enough that Level 1 already describes them adequately, which is itself a defensible design statement.

4.1 Process 4.0 — AI Matching & Scoring

The scoring engine, decomposed. Note that it is driven by a job-published event rather than by a user request: scoring runs asynchronously in a worker, so the recruiter's shortlist page reads precomputed rows rather than triggering a million comparisons.



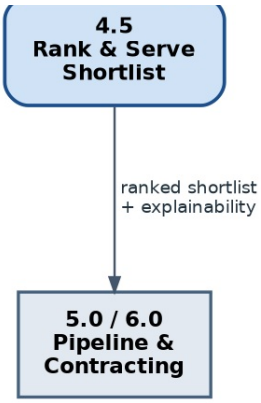


Figure 3 — Level 2 decomposition of Process 4.0

No.	Sub-process	Transformation performed
4.1	Build candidate pool	Hard-filters on mandatory skills, location and availability so scoring runs on hundreds of candidates rather than the full database
4.2	Extract features	Reads parsed resume text, skill proficiency, experience and education into a comparable feature vector
4.3	Apply weightage formula	Combines features using the per-job weights from D3, producing a 0–100 score
4.4	Persist score	Writes score plus per-factor breakdown; marks the prior score for that pair as superseded
4.5	Rank and serve	Returns the ordered shortlist with explainability data to processes 5.0 and 6.0

4.2 Process 5.0 — Full-Time Recruitment Pipeline

The permanent-hiring flow, from outbound invitation through to a signed offer. The two entry points — a candidate applying directly, or a recruiter inviting — converge on the same application record, distinguished only by its source.

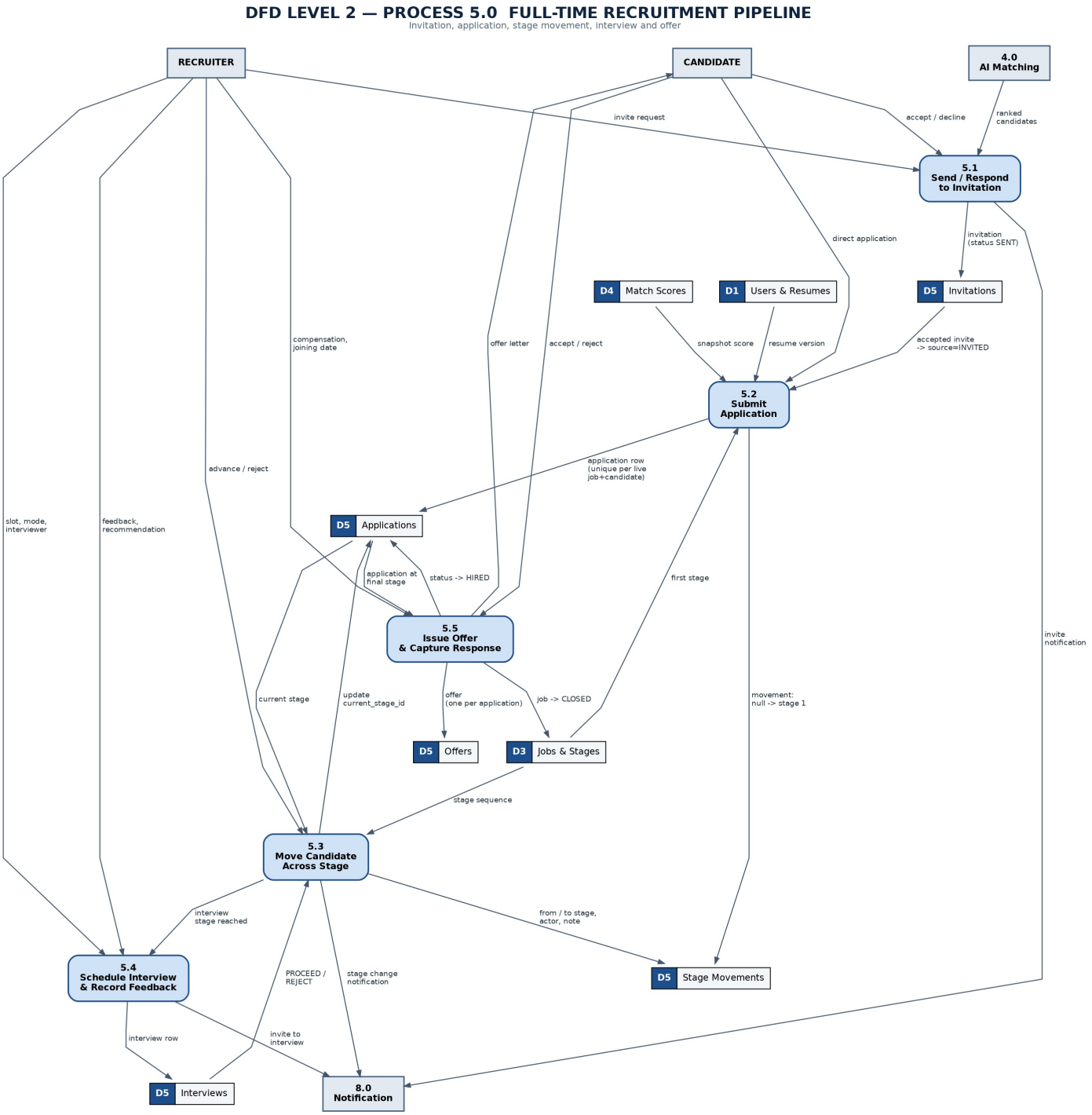
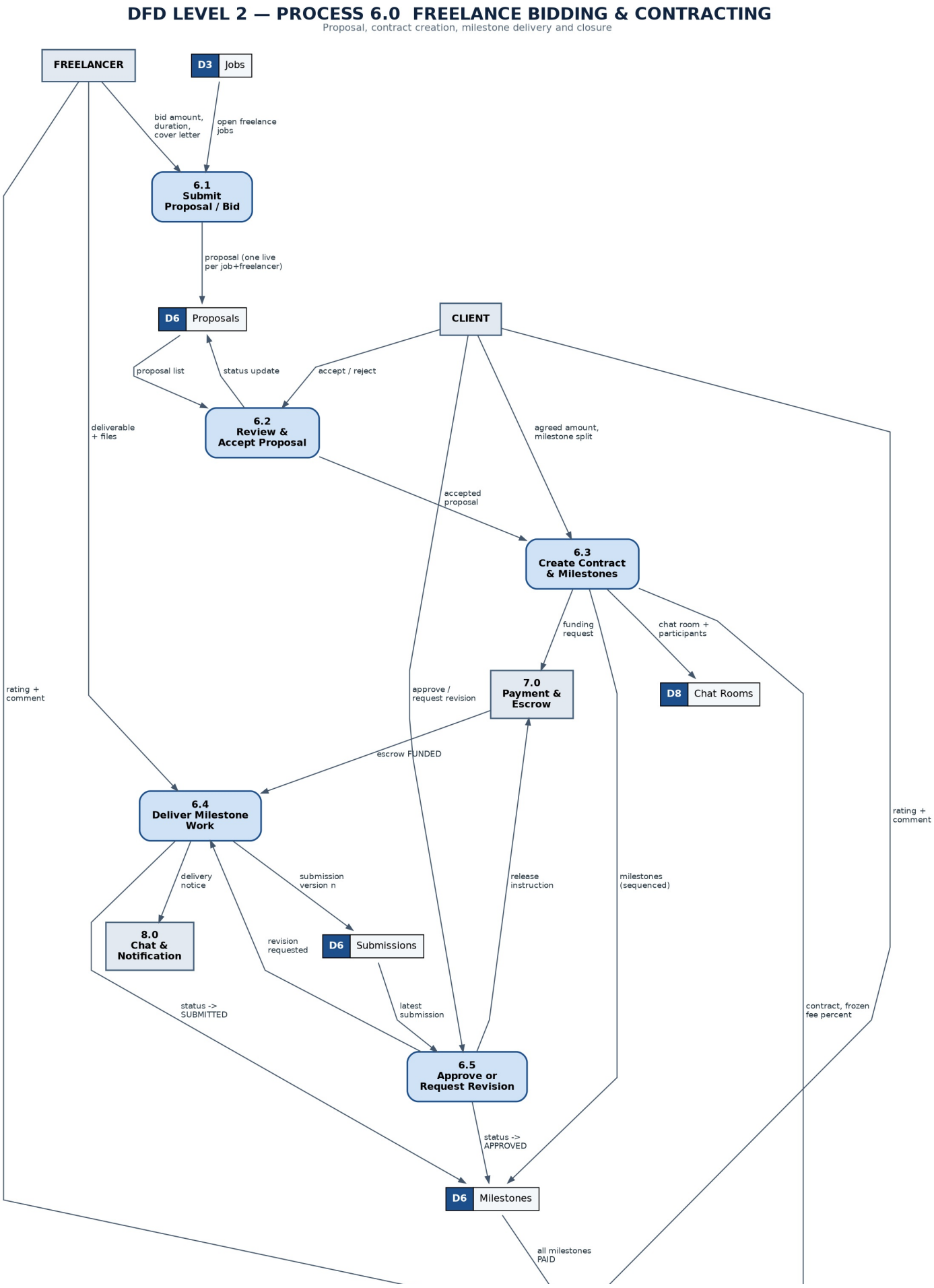


Figure 4 — Level 2 decomposition of Process 5.0

No.	Sub-process	Transformation performed
5.1	Send / respond to invitation	Creates outbound invitations from the ranked list; records acceptance or decline
5.2	Submit application	Creates the application, pins the resume version and score snapshot, writes the opening stage movement
5.3	Move across stage	Appends to the movement history and updates the cached current stage — the two writes are one transaction
5.4	Schedule interview	Creates the interview against a stage, then captures structured feedback and a recommendation
5.5	Issue offer	Generates the offer, captures the response, closes the application and the job

4.3 Process 6.0 — Freelance Bidding & Contracting

The commercial half of the platform. The critical transition is 6.2 → 6.3: acceptance of a proposal is the moment a bid becomes a binding contract, and it is where the milestone structure, chat room and frozen fee percentage all come into existence.



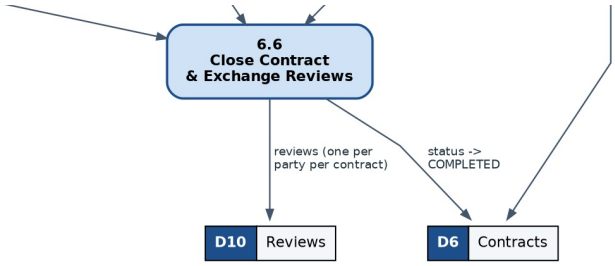


Figure 5 — Level 2 decomposition of Process 6.0

No.	Sub-process	Transformation performed
6.1	Submit proposal	Records a bid with amount, duration and pitch; one live proposal per freelancer per job
6.2	Review and accept	Client shortlists, rejects or accepts; acceptance triggers contract creation
6.3	Create contract	Creates contract with agreed amount and frozen fee, sequenced milestones, and the scoped chat room
6.4	Deliver milestone	Records a versioned submission with attachments; never overwrites a prior version
6.5	Approve or revise	Either loops back for revision or approves, which instructs 7.0 to release escrow
6.6	Close and review	Marks the contract complete and records one review per party

4.4 Process 7.0 — Payment, Escrow & Payout

The most correctness-critical decomposition, and the one worth knowing best. Three details in this diagram map onto real safeguards: 7.3 is idempotent because the gateway reference is unique, 7.5 reads the wallet under a row lock, and 7.7 freezes escrow the moment a dispute is raised.

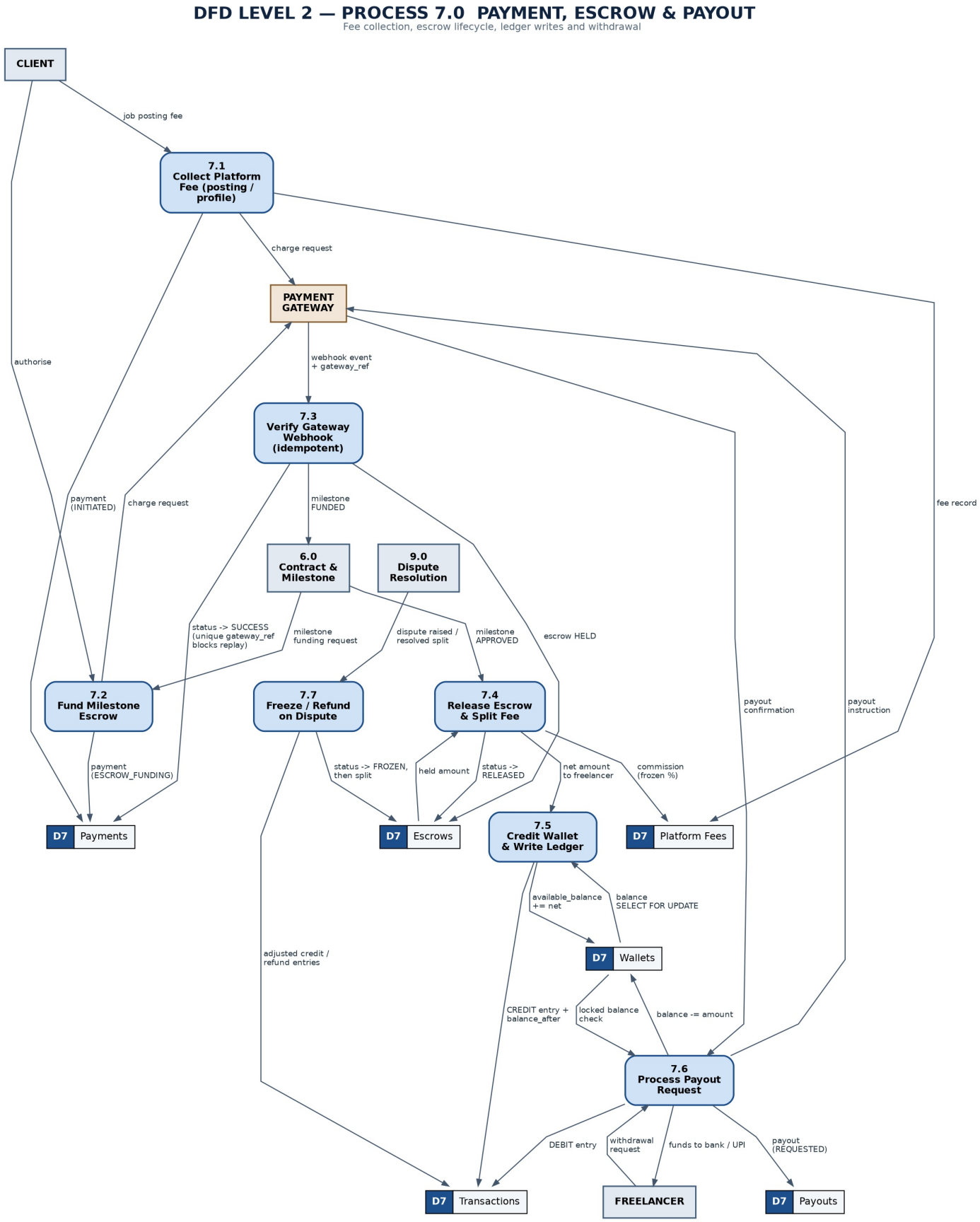


Figure 6 — Level 2 decomposition of Process 7.0

No.	Sub-process	Transformation performed
7.1	Collect platform fee	Charges job-posting and candidate-profile fees; records the revenue line
7.2	Fund escrow	Initiates the client's charge for one milestone
7.3	Verify webhook	Confirms payment; the unique gateway reference makes a replayed webhook a no-op
7.4	Release and split	Releases held funds, deducts commission at the contract's frozen percentage
7.5	Credit wallet, write ledger	Locks the wallet row, credits the net amount, appends the ledger entry with running balance
7.6	Process payout	Validates against the locked balance, debits, and instructs the gateway
7.7	Freeze / refund	Freezes escrow on dispute; applies the admin's split on resolution

5. Data Dictionary — Key Flows

A DFD is conventionally accompanied by a data dictionary defining the composition of each flow. The notation is: `=` composed of, `+` and, `{ }` repeating group, `( )` optional, `[ | ]` either-or.

Data flow	Composition
job posting	title + description + job_type[FULL_TIME|FREELANCE] + (salary_min + salary_max) + (budget + budget_type) + location + {required_skill} + deadline
ai weightage	skill_weight + experience_weight + education_weight + project_weight (sum = 100)
match score	job_id + candidate_id + score + breakdown{factor + sub_score} + model_version + is_current
application	job_id + candidate_id + resume_id + source[DIRECT|INVITED] + snapshot_ai_score + applied_at
stage movement	application_id + (from_stage) + to_stage + moved_by + (note) + moved_at
proposal	job_id + candidate_id + bid_amount + duration_days + cover_letter + status
contract	proposal_id + client_id + freelancer_id + agreed_amount + platform_fee_percent + {milestone}
milestone	contract_id + title + amount + sequence + (due_date) + status
escrow instruction	milestone_id + amount + currency + [FUND|RELEASE|REFUND|FREEZE]
ledger entry	wallet_id + direction[CREDIT|DEBIT] + amount + balance_after + reference_type + reference_id
dispute ruling	dispute_id + admin_id + resolution_note + amount_to_freelancer + amount_to_client

6. Balancing — Proving the Levels Agree

Balancing is the formal consistency check between levels, and a favourite examination question. The rule: **every flow entering or leaving a parent process must appear in its child diagram.**

Level 0 flow	Appears in Level 1 at	Appears in Level 2 at
Recruiter → job postings	Recruiter → 3.0	—
Candidate → proposals	Candidate → 6.0	Freelancer → 6.1
Recruiter → escrow funding	Recruiter → 7.0	Client → 7.2
System → payment gateway	7.0 → Payment Gateway	7.1, 7.2, 7.6 → Gateway
Gateway → confirmation	Payment Gateway → 7.0	Gateway → 7.3
System → candidate payout	7.0 → Candidate	7.6 → Freelancer
Admin → dispute rulings	Admin → 9.0	9.0 → 7.7

> **An unbalanced DFD is the most commonly penalised error.** If Level 0 shows six flows to the payment gateway and Level 1 shows four, the diagrams contradict each other. Check each level against its parent before submitting, and be ready to trace one flow all the way from Level 0 down to Level 2 on request — that is the standard viva question.

7. Interview & Viva Questions

1. What is the difference between a DFD and a flowchart?

A DFD shows *data* movement and transformation, with no notion of sequence or decisions. A flowchart shows *control* flow — order of execution, branches, loops. There are no if-then-else symbols in a DFD, and no timing. That is why my milestone approval appears as a data flow into process 7.4 rather than as a decision diamond.

2. Why does your Level 0 have no data stores?

Because a context diagram shows only what crosses the system boundary. All storage is internal, so at that abstraction it is invisible. Stores first appear at Level 1, where the system is opened up into its constituent processes.

3. What is balancing, and is your diagram balanced?

Balancing means a child diagram's inputs and outputs must exactly match its parent process's. Yes — for example the escrow funding flow enters at Level 0 from the client, attaches to process 7.0 at Level 1, and resolves to sub-process 7.2 at Level 2. Nothing appears or disappears between levels.

4. Why can't two data stores connect directly?

Because a store is passive — it has no capability to act. Moving data from one store to another is a transformation, and every transformation must be a numbered process. A direct store-to-store arrow is formally invalid.

5. How did you decide which processes to decompose to Level 2?

By internal complexity. Processes 4, 5, 6 and 7 have multi-step internal logic worth exposing — scoring has five distinct stages, and the payment process has seven with real correctness constraints. Processes like authentication are essentially single transformations, so decomposing them would produce a diagram that restates its own parent. Decomposing everything to the same depth is a common mistake; depth should follow complexity.

6. How do your data stores relate to your database tables?

Each store is a logical grouping of tables that processes read and write as a unit. D1 covers eight tables — users, profiles, companies, resumes, portfolios — because from a data-movement view they behave as one. The mapping is documented, which is how the DFD and ER diagram stay consistent with each other.

7. Trace the flow of money through your DFD.

The client sends escrow funding to 7.2, which requests a charge from the gateway. The gateway's webhook enters 7.3, which verifies it idempotently using the unique gateway reference and writes the escrow as HELD. When 6.5 approves a milestone, it instructs 7.4, which releases the escrow and deducts the commission frozen on the contract. The net amount passes to 7.5, which locks the wallet row, credits it and appends a ledger entry. Later 7.6 handles withdrawal, debiting the wallet and instructing the gateway to pay out.

8. Where does the AI matching sit, and why is it a separate process?

Process 4.0, sitting between job management and both hiring flows. It is separate because it is triggered by a job-published event rather than a user request, and it runs asynchronously in a worker — scoring is CPU-bound and cannot block the request path. The recruiter's shortlist reads precomputed rows from D4, which is why 4.5 is a distinct sub-process from 4.3.

9. Is a dispute a process or a data flow?

Both, at different levels. Raising a dispute is a data flow from the candidate into process 9.0. Resolving it is work performed by 9.0, which emits a split instruction as a data flow into 7.7. The dispute record itself is data at rest in store D11. Distinguishing the three roles a concept can play is exactly what DFD notation is for.

10. What would change in this DFD if you added video interviews?

A new external entity for the video provider, a flow from 5.4 to it carrying meeting requests, a return flow carrying join links and recordings, and an extension of D5 to hold the recording reference. Nothing else changes — which is a reasonable argument that the decomposition boundaries were drawn in the right places.

Preparation advice. The two things you will most likely be asked to do live are *state the difference between a DFD and an ER diagram*, and *trace one flow from Level 0 down to Level 2*. Practise the money trace in question 7 out loud — it touches four sub-processes, two external entities and three data stores, and demonstrates that you understand your own diagram rather than having drawn it from a template.